BaezEshani-FinalCheckIn

Eshani Baez

2026-09-30

Data Source: For your final project, you will be working with real RNA-seq count and meta data from breast cancer patients enrolled in The Cancer Genome Atlas (TCGA) Breast Invasive Carcinoma (TCGA-BRCA) cohort. This data encompasses diverse subtypes, tumor stages, and patient demographics. Some samples represent tumor tissues; others matched normal control samples from the same patient. TCGA is a landmark cancer genomics program jointly funded by the National Cancer Institute (NCI) and the National Human Genome Research Institute (NHGRI). TCGA generated comprehensive molecular profiles for over 11,000 patients across 33 cancer types, including bulk RNA-seq DNA methylation, copy number variation, somatic mutations, and clinical annotation.

This data source includes the following meta data: • Patient Demographics • Tumor Characteristics • Treatment Information • Survival Data • Sample Details

1) Data Preparation:

a) Download the Sample RNA-seq Count Matrix and associated Metadata. i) Ensure that you have both the count matrix (counts.csv) and the metadata file (meta_data.csv) available for your analysis.

```
# EXPLANATION:
# Sets the working directory to that of the current working markdown file as
# well as any data files to reference in this script. The two data files
# ("meta_data.csv" and "counts.csv") are then read into the document from the
# working directory in order to clean and run analyses on the raw data.

# All necessary libraries and packages are loaded for use throughout the script
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ComplexHeatmap)
```

```
## Loading required package: grid
```

```
## ========================================
## ComplexHeatmap version 2.28.0
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##     genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## ========================================
```

```
library(circlize)
```

```
## ========================================
## circlize version 0.4.18
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## ========================================
```

```
library(ggpubr)

# Returns R and package versions used throughout the current markdown file
# sessionInfo()

setwd("/Users/eshani/Downloads/Dartmouth MS/Semester 1/Foundations in Data Science (HSE711)/HSE711 - Fi
meta_data <- read_csv("meta_data.csv")
```

```
## Rows: 1231 Columns: 98
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (72): barcode, patient, sample, shortLetterCode, definition, sample_subm...
## dbl (16): days_to_collection, initial_weight, days_to_diagnosis, age_at_diag...
## lgl (10): oct_embedded, is_ffpe, diagnosis_is_primary_disease, age_is_obfusc...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
counts_data <- read_csv("counts.csv")
```

```
## New names:
## Rows: 60660 Columns: 1232
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (1): ...1 dbl (1231): TCGA-GM-A2DL-01A-11R-A18M-07,
## TCGA-AC-A2QI-01A-12R-A19W-07, TCGA...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

2) Gene Selection and Summary Statistics:

a) Select One Gene: choose a gene from the dataset that interests you.
b) Generate Summary Statistics: Using the count data from the selected gene, compute and report summary statistics, such as mean, median, standard deviation, minimum, and maximum.

```r
# EXPLANATION:
# The raw counts data is in wide form with each sample's counts in columns and
# genes as rows. The below, first transposes the data frame so that genes become
# columns and samples become rows.

# CITATION (for transposing the raw data):
# Pivoting. (n.d.). Retrieved September 30, 2026, from https://tidyr.tidyverse.org/articles/pivot.html#

# Transposes the counts_data so that ensembl id genes become column names with
# their associated values/counts for each sample. Sample names have become row titles.
counts_data_t <- counts_data |>
  pivot_longer(- ...1, names_to = "barcode", values_to = "value") |>
  pivot_wider(names_from = ...1, values_from = value)

# EXPLANATION:
# Creates a function that takes in a gene counts data set (where each column is a gene's
# ensembl ID and each row is a different sample), a specific ensembl ID from the data set,
# a table theme (from ggpubr's table themes), a title for the summary statistics table it
# will generate, and a name for the .png file it will save of the table. The function will
# first generate the following summary statistics for the selected gene: mean, median,
# minimum, maximum, and standard deviation. Then, the summary will be formatted into a
# table using the ggpubr package. Ultimately ggsave is used to save the table to a .png
# file with the file name of your choosing.

# CITATION:
# https://rpkgs.datanovia.com/ggpubr/reference/ggtexttable.html

summary_stats_table <- function(data_set, ensembl_id, table_theme, table_title, file_name) {
  # Generates the summary statistics for ensembl_id
  stats <- data_set |> summarise(
    Mean = round(mean(.data[[ensembl_id]]), 2),
    Median = round(median(.data[[ensembl_id]]), 2),
    Min = round(min(.data[[ensembl_id]]), 2),
    Max = round(max(.data[[ensembl_id]]), 2),
```

```
    SD = round(sd(.data[[ensembl_id]]), 2)
  )

  # Creates a table of the summary statistics.
  stats_table <- ggtexttable(
    stats,
    rows = NULL,
    theme = ttheme(table_theme, base_size = 16)
  ) |>
    tab_add_title(
      text = table_title,
      face = "bold",
      size = 14,
      just = "center"
    )

  # Saves plot as .png for presentation
  ggsave(file_name, plot = stats_table, width = 6, height = 2, dpi = 300)

  return(stats_table)
}

# Creates a BRCA1 table of summary statistics by calling the summary_stats_table function
BRCA1_table <- summary_stats_table(
  data_set = counts_data_t,
  ensembl_id = "ENSG00000012048.23",
  table_theme = "mBlue",
  table_title = "BRCA1 Expression Summary Statistics",
  file_name = "BRCA1_stats_table.png"
)

BRCA1_table
```

BRCA1 Expression Summary Statistics

Mean	Median	Min	Max	SD
1304.21	919	21	8050	1137.35

```
# Creates a PALB2 table of summary statistics by calling the summary_stats_table function
PALB2_table <- summary_stats_table(
  data_set = counts_data_t,
  ensembl_id = "ENSG00000083093.10",
  table_theme = "mBlue",
  table_title = "PALB2 Expression Summary Statistics",
  file_name = "PALB2_stats_table.png"
)

PALB2_table
```

PALB2 Expression Summary Statistics

Mean	Median	Min	Max	SD
1167.13	1040	142	4389	599.81

3) Visualization:

a) Create a Histogram: Use ggplot2 to generate a histogram of the count data for the selected gene.This visualization should effectively display the distribution of the counts.

```
# EXPLANATION:
# Creates a histogram of BRCA1 Gene counts across the TCGA-BRCA cohort samples.
# Bin width has been adjusted to display the counts distribution. Bins have been
# outlined in black for improved readability.

# Calculates mean and median for BRCA1 counts across samples
BRCA1_mean <- mean(counts_data_t$ENSG00000012048.23)
BRCA1_median <- median(counts_data_t$ENSG00000012048.23)

BRCA1_hist <- ggplot(counts_data_t, aes(x = ENSG00000012048.23)) +
  geom_histogram(binwidth = 150, fill = "deeppink3", color = "black") +
  labs(title = "BRCA1 Gene Count Across 1,231 Samples",
       x = "BRCA1 Gene (ENSG00000012048.23) Count",
       y = "Frequency") +
  # Adds a vertical line at BRCA1 mean
  geom_vline(xintercept = BRCA1_mean, color = "#FF00CC") +

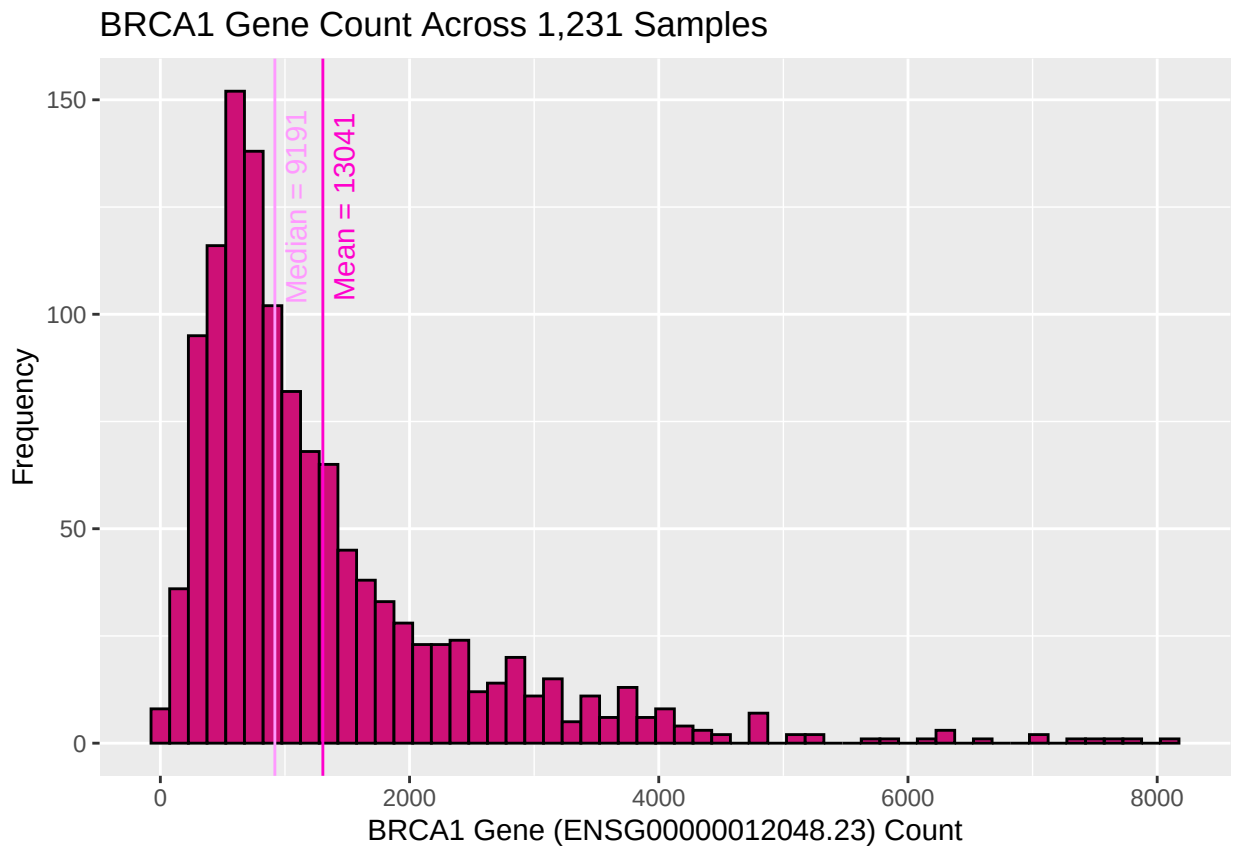
  # Adds a label to the mean line
  annotate("text", x = BRCA1_mean, y = 125,
           label = paste0("Mean = ", round(BRCA1_mean), 1),
```

```
           color = "#FF00CC",
           angle = 90, vjust = 1.5) +

  # Adds a vertical line at BRCA1 median
  geom_vline(xintercept = BRCA1_median, color = "#FF99FF") +

   # Adds a label to the median line
  annotate("text", x = BRCA1_median, y = 125,
           label = paste0("Median = ", round(BRCA1_median), 1),
           color = "#FF99FF",
           angle = 90, vjust = 1.5)

BRCA1_hist
```




```
ggsave("BRCA1_histogram.png", plot = BRCA1_hist) # Saves plot as .png for presentation
```

```
## Saving 6.5 x 4.5 in image
```

b) Create a Scatter Plot: Select a second gene from the dataset. Create a scatter plot using ggplot2 to compare the count data of the two selected genes.

```
# EXPLANATION:
# Generates a scatterplot comparing BRCA1 (ENSG00000012048.23) and PALB2 (ENSG00000083093.10)
# counts in the TCGA-BRCA cohort samples. BRCA1 gene counts are plotted on the x-axis
```

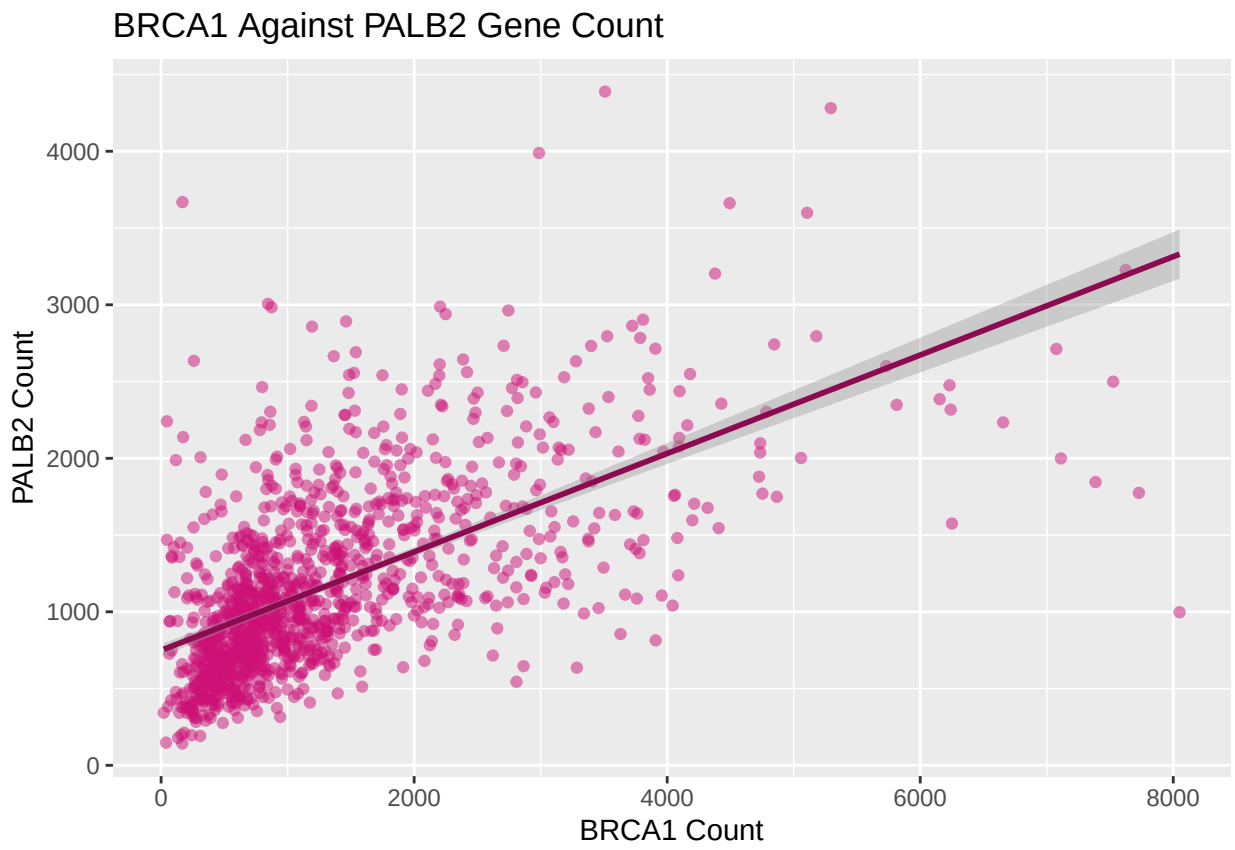
```r
# and PALB2 gene counts are plotted on the y-axis. Transparency is adjusted for
# the points, to visually observe overlap and where most points are clumping.

# Assigns ENSG00000012048.23 and ENSG00000083093.10 for analysis in the ggplot
BRCA1_PALB2_scat <- ggplot(counts_data_t, aes(x = ENSG00000012048.23, y = ENSG00000083093.10)) +

  # Applies a level of transparency to the plot points to visualize where majority
  # of the data is saturated
  geom_point(color = "deeppink3", alpha = 0.5) +

  labs(x = "BRCA1 Count", y = "PALB2 Count", title = "BRCA1 Against PALB2 Gene Count") +
  geom_smooth(method = "lm", formula = y ~ x, se = TRUE, color = "deeppink4")   # Adds regression line

BRCA1_PALB2_scat
```




```r
# Saves plot as .png for presentation
ggsave("BRCA1_PALB2_scatterplot.png", plot = BRCA1_PALB2_scat)
```

```
## Saving 6.5 x 4.5 in image
```

c) Create a Violin Plot: Select one covariate from your metadata. Using the count data from the first gene and the selected covariate, generate a violin plot that illustrates the distribution of count data based on the covariate. For example, if you choose "primary_diagnosis", your plot should display a violin plot for each level in "primary_diagnosis".

```r
# EXPLANATION
# The below, generates a violin plot of BRCA1 gene count across all TCGA-BRCA cohort
# samples. Counts are grouped by each participant's age at diagnosis as grouped by decade.
# Ages at diagnosis in the raw meta_data dataset are first replaced by age in years
# rather than age in days. Next, a new, identical data frame (df) is created that
# groups ages into ranges by decade (20s, 30s, 40s, etc.). The age categorization is
# used for creating violin plots for each "decade".

# In order to create a plot with data from both the counts df and the meta_data df,
# the two were merged by their shared sample "barcode" column.

# Ggplot was then used to generate a violin plot with age at diagnosis to generate
# multiple violin plots on the x-axis and BRCA1 count on the y-axis. Samples that did
# not include age data/were "NA" were filtered out. Each age group's plot has a unique
# color. Boxplots have been added over the violin plots for each age group to cleanly,
# visually, summarize each group.

# I also ran a quick table function to check that my age categorization worked as I
# had hoped and for further analysis of the age at diagnosis data.


meta_data_age_cat <- meta_data |>
  # Adjust age values to be calculated in approximate years rather than days,
  # rounded to 1 decimal point. Replace the age_at_diagnosis column in the meta_data df.
  mutate(age_at_diagnosis = round(age_at_diagnosis/365, 1)) |>

  # Categorizes participant age at diagnosis for each sample into every 10 year age group
  mutate(age_at_diagnosis = case_when(age_at_diagnosis >= 20 & age_at_diagnosis < 30 ~ "20s",
                   age_at_diagnosis >= 30 & age_at_diagnosis < 40 ~ "30s",
                   age_at_diagnosis >= 40 & age_at_diagnosis < 50 ~ "40s",
                   age_at_diagnosis >= 50 & age_at_diagnosis < 60 ~ "50s",
                   age_at_diagnosis >= 60 & age_at_diagnosis < 70 ~ "60s",
                   age_at_diagnosis >= 70 & age_at_diagnosis < 80 ~ "70s",
                   age_at_diagnosis >= 80 & age_at_diagnosis < 90 ~ "80s",
                   age_at_diagnosis >= 90 ~ "90s+"
                   ))

# Merges the transposed counts and age categorized meta_data data frames by sample barcode
# DFs are in the same wide format with samples as rows and genes/observations as columns
all_BRCA_data <- full_join(x = meta_data_age_cat, y = counts_data_t, by = "barcode")

# Returns groups and their counts within the age at diagnosis column in the categorized meta data df
table(meta_data_age_cat$age_at_diagnosis)
```

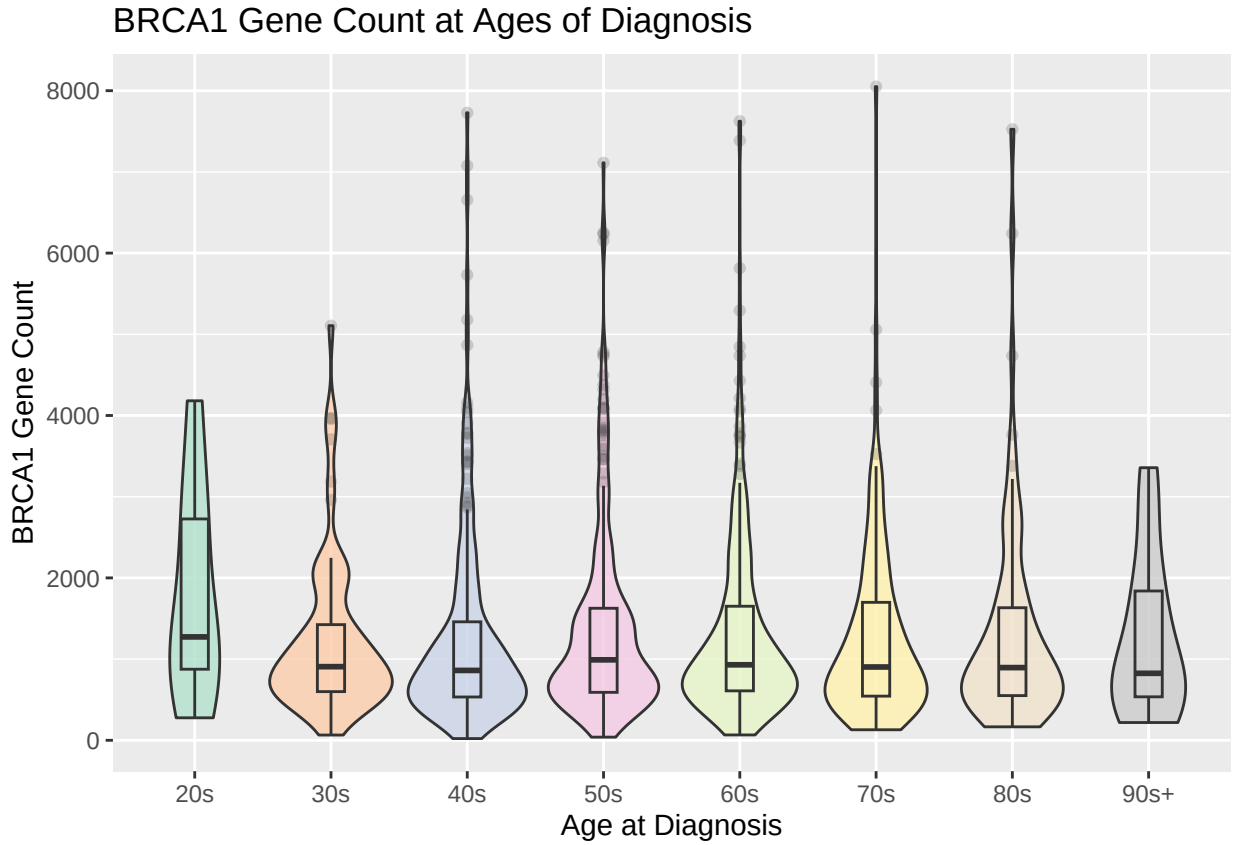
```
##
##  20s  30s  40s  50s  60s  70s  80s 90s+
##    8   71  238  307  309  161   70   12
```

```r
# Violin plot of BRCA1 gene count by age of diagnosis
BRCA1_violin <- ggplot(all_BRCA_data |> filter(!is.na(age_at_diagnosis)),
                       aes(y = ENSG00000012048.23,
                           x = age_at_diagnosis,
                           fill = age_at_diagnosis)) +
```

```
  geom_violin(alpha = 0.8) +
  geom_boxplot(alpha = 0.2, width = 0.2) +

  # Adds color and removes the legend as it is redundant with the x-axis label
  scale_fill_brewer(palette = "Pastel2", guide = "none") +
  labs(x = "Age at Diagnosis", # Adds axes labels and a title to the violin plot
       y = "BRCA1 Gene Count",
       title = "BRCA1 Gene Count at Ages of Diagnosis")

BRCA1_violin
```




```
ggsave("BRCA1_violinplot.png", plot = BRCA1_violin) # Saves plot as .png for presentation
```

```
## Saving 6.5 x 4.5 in image
```

d) Heatmap Analysis: i) Select 10 genes: Choose a set of 10 different genes from the count matrix for your heatmap. ii) Generate a Heatmap: Use the ComplexHeatmap package in R to create a heatmap of the count data for the selected genes. iii) Add an Annotation Bar: Include an annotation bar reflecting your chosen covariate for further context and interpretation of the data.

```
# EXPLANATION
# Uses the ComplexHeatmap package to create a heatmap of 10 select genes from the
# TCGA-BRCA counts data frame. The selected genes were as follows: BRCA1 (ENSG00000012048.23),
# BRCA2 (ENSG00000139618.16), PALB2 (ENSG00000083093.10), ATM (ENSG00000149311.19),
```

```r
# BRIP1 (ENSG00000136492.9),  BARD1 (ENSG00000138376.11), RAD50 (ENSG00000113522.14),
# RAD51C (ENSG00000108384.15), CHEK2 (ENSG00000183765.23), NBN (ENSG00000104320.14).
# Counts data associated with the 10 selected genes is pulled from the df by the ensembl
# IDs which had been previously assigned as row names. Normalized expressions/gene count
# values are organized into age at diagnosis group (rather than clustered by similarity),
# making it visually straightforward to compare whether gene expression patterns differ
# across age decades while still preserving gene-level clustering (on the left) to reveal
# which genes behave similarly.

# Used stack overflow to learn how to save heatmaps to .png files, as ggsave() is a ggplot object
#CITATION: Ann_K. (2022, Feb 4). Re: How to save a heat map created using the ComplexHeatmap
# package to png format? [Online forum post]. Stack Overflow. https://stackoverflow.com/questions/66044

# Creates new version of counts_data using ensembl IDs under "...1" header as row names
counts_data_gene_rownames <- counts_data |>
  column_to_rownames(var = "...1")

select_genes <- c("ENSG00000012048.23", "ENSG00000139618.16", "ENSG00000083093.10",
                  "ENSG00000149311.19", "ENSG00000136492.9", "ENSG00000138376.11",
                  "ENSG00000113522.14", "ENSG00000108384.15", "ENSG00000183765.23",
                  "ENSG00000104320.14")

# Subsets rows by name for the 10 selected genes
genes_subset <- counts_data_gene_rownames[select_genes, ]

# Converts the subset data to a matrix, since Heatmap() requires a matrix datatype for analysis
gene_matrix <- as.matrix(genes_subset)

# Look up each sample's age group, then drop samples with missing age
age_annotation_data <- meta_data_age_cat$age_at_diagnosis[match(colnames(gene_matrix),
                                                                meta_data_age_cat$barcode)]

keep <- !is.na(age_annotation_data)
gene_matrix <- gene_matrix[, keep]
age_annotation_data <- age_annotation_data[keep]

# Normalizes each row independently for general comparison between genes using z-scores.
gene_matrix_scaled <- t(scale(t(gene_matrix)))

# Creates gene labels that are human readable by mapping the ensembl IDs found in the raw
# data to their associated gene symbols. Ensembl rownames are replaced in the df by
# readable labels.
gene_symbol_map <- c("ENSG00000012048.23" = "BRCA1", "ENSG00000139618.16" = "BRCA2",
                     "ENSG00000083093.10" = "PALB2", "ENSG00000149311.19" = "ATM",
                     "ENSG00000136492.9" = "BRIP1", "ENSG00000138376.11" = "BARD1",
                     "ENSG00000113522.14" = "RAD50", "ENSG00000108384.15" = "RAD51C",
                     "ENSG00000183765.23" = "CHEK2", "ENSG00000104320.14" = "NBN")
rownames(gene_matrix_scaled) <- gene_symbol_map[rownames(gene_matrix_scaled)]

# Creates a Heatmap annotation bar reflecting ages at diagnosis. Each sample is associated
# with a color in the age bar (including samples with missing age data)
```

```r
col_annotation <- HeatmapAnnotation(
  AgeGroup = age_annotation_data,
  col = list(AgeGroup = c("20s" = "#BCC69B",
                          "30s" = "#3D3B8E",
                          "40s" = "#6883BA",
                          "50s" = "#B1BEDA",
                          "60s" = "#6EB4D1",
                          "70s" = "#E072A4",
                          "80s" = "#C8AA9E",
                          "90s+" = "#B0E298")
  )
)

# Works with draw() to save Heatmap as .png for presentation.
png("genes_heatmap.png", width = 10, height = 8, units = "in", res = 300)

# Creates a heatmap of 10 selected genes
genes_heatmap_by_age <- Heatmap(gene_matrix_scaled,
  name = "Z-Score",
  # Defines the color gradient for z-scores with smooth interpolation between -2 and 2
  col = colorRamp2(c(-2, 0, 2), c("navy", "white", "firebrick")),
  top_annotation = col_annotation, # Attaches the annotation bar created above
  show_row_names = TRUE,
  show_column_names = FALSE,
  cluster_columns = FALSE, # Turns off top dendrogram as it is not relevant to current analysis
  cluster_rows = TRUE, # Groups genes with similar expression patterns
  column_title = "Gene Expression by Age at Diagnosis",
  column_order = order(age_annotation_data), # Orders sample z-score data by age group
  row_title = "Genes",
  heatmap_legend_param = list(title = "Z-Score")
  )



draw(genes_heatmap_by_age)
dev.off()
```

```
## pdf
##   2
```

```r
draw(genes_heatmap_by_age)
```




```
# EXPLANATION: Compares gene expression across the three sample types in the TCGA-BRCA
# cohort (primary solid tumor, solid tissue normal, and metastatic). It first checks
# the sample type column for missing values, then defines a reusable function that
# builds a boxplot + jitter plot of log2-transformed counts for one gene. with Wilcoxon
# rank-sum p-values added. The function is applied to BRCA1, BRCA2, and PALB2, and the
# three plots are arranged side-by-side in one multi-panel figure.

# Count how many samples fall into each sample type (definition column)
table(meta_data$definition)
```

```
##
##          Metastatic Primary solid Tumor Solid Tissue Normal
##                   7                1111                 113
```

```
# Checks for any missing sample type values
sum(is.na(meta_data$definition))
```

```
## [1] 0
```

```
# Function: build a boxplot + jitter plot of one gene's expression by sample type
# data_set      - df with one row per sample (counts joined to metadata)
# ensemble_ID   - column name of the gene's Ensembl ID (counts column)
# gene_name     - readable gene name used in axis label and title
# metadata_col  - metadata column to group by (here, "definition")
```

```r
sample_type_jitter_plot <- function(data_set, ensembl_ID, gene_name, metadata_col) {

  # Removes samples with missing sample types, then log2-transform the counts
  sample_types <- data_set |>
  filter(!is.na(.data[[metadata_col]])) |>
  mutate(gene_log = log2(.data[[ensembl_ID]] + 1))

# Main base plot: x = sample type, y = log2 expression, fill = sample type
p <- ggplot(sample_types,
            aes(x = .data[[metadata_col]], y = gene_log, fill = .data[[metadata_col]])) +
  # Boxplot summarizing median and spread per group
  geom_boxplot(alpha = 0.5) +

  # Individual samples overlaid, spread horizontally to reduce overlap
  geom_jitter(width = 0.15, alpha = 0.4, size = 1) +

  # Tilt x-axis labels 45 degrees to fit the longer names, centers the title
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        plot.title = element_text(hjust = 0.5)) +

  # Adds Wilcoxon rank-sum p-values for each pair of sample types
  stat_compare_means(
    comparisons = list(
      c("Primary solid Tumor", "Solid Tissue Normal"),
      c("Primary solid Tumor", "Metastatic"),
      c("Solid Tissue Normal", "Metastatic")
    ),
    method = "wilcox.test",
    label = "p.format") +

  scale_fill_brewer(palette = "Pastel2", guide = "none") +

  # Axis labels and wrapped title
  labs(x = "Sample Type",
       y = paste("log2(", gene_name, "Count + 1)"),
       title = str_wrap(paste(gene_name, "Expression by Sample Type"), width = 25),
       subtitle = "(Pairwise Wilcoxon rank-sum tests)")

return(p)
}

# Creates one plot per gene (Ensembl ID, display name, grouping column)
BRCA1_jitter <- sample_type_jitter_plot(all_BRCA_data, "ENSG00000012048.23", "BRCA1", "definition")


## `stat_compare_means()` with `comparisons` displays *unadjusted* p-values (no correction for multiple
## i For p-values adjusted for multiple comparisons, use `geom_pwc()`, or `stat_pvalue_manual()` togeth
## This message is displayed once per session.

BRCA2_jitter <- sample_type_jitter_plot(all_BRCA_data, "ENSG00000139618.16", "BRCA2", "definition")
PALB2_jitter <- sample_type_jitter_plot(all_BRCA_data, "ENSG00000083093.10", "PALB2", "definition")

# Arranges the three single plots in a single row (3 columns x 1 row)
jitter_plots_arranged <- ggarrange(ncol = 3, nrow = 1, widths = 2, heights = 6, plotlist = BRCA1_jitter
```

```r
ggsave("jitter_plots.png", plot = jitter_plots_arranged, width = 12, height = 8)
```

```r
# LIMITATION: Determines how many unique patients/patient codes exist in the
# data to confirm if there are mutliple samples included from the same patient.
n_distinct(meta_data$patient)
```

```
## [1] 1095
```

```r
# LIMITATION: Determines how many of each sample type exist in the data set.
table(meta_data$definition)
```

```
##
##          Metastatic Primary solid Tumor Solid Tissue Normal
##                   7               1111                113
```

```r
# LIMITATION: Determines how many of each sample type exist in the data set.
table(meta_data_age_cat$age_at_diagnosis)
```

```
##
##  20s  30s  40s  50s  60s  70s  80s 90s+
##    8   71  238  307  309  161   70   12
```

```r
# LIMITATION: Determines how many samples are missing an age at diagnosis.
sum(is.na(meta_data$age_at_diagnosis))
```

```
## [1] 55
```